LECTURE 2
INTRODUCTION

CEN 348- Artificial Intelligence Systems

Department of Computer Engineering, Cukurova University

Assist.Prof.Dr. Ezgi ZORARPACI
E-mail: ezorarpaci@gmail.com

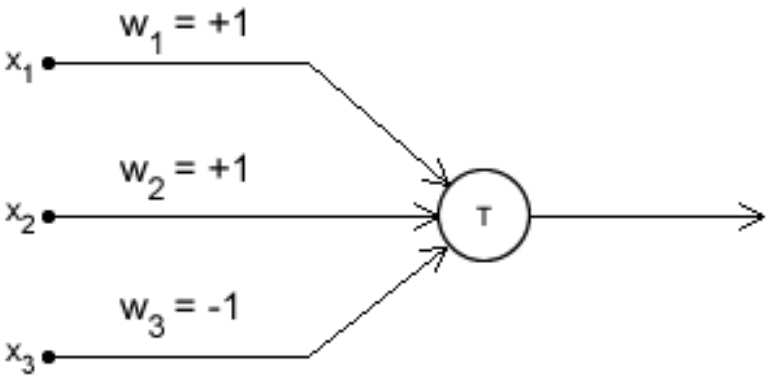
M-P neuron learning model

$$g(x_1, x_2, x_3, \ldots, x_n) = g(\mathbf{x}) = \sum_{i=1}^{n} x_i$$

$$\begin{aligned} y = f(g(\mathbf{x})) &= 1 \quad if \quad g(\mathbf{x}) \geq \theta \\ &= 0 \quad if \quad g(\mathbf{x}) < \theta \end{aligned}$$

- Learning in MP Neuron consists of finding the threshold value with lowest error for prediction.

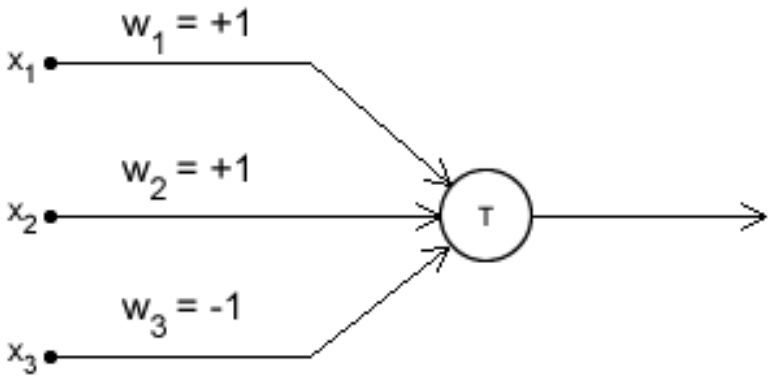
M-P neuron learning model



- The McCulloch-Pitts model was an extremely simple artificial neuron.
- The inputs could be either a zero or a one.
- The output was a zero or a one.
- Each input could be either excitatory or inhibitory.
- Now the whole point was to sum the inputs.

- If an input is one, and is excitatory in nature, it added one.
- If it was one, and was inhibitory, it subtracted one from the sum.
- This is done for all inputs, and a final sum is calculated.
- Now, if this final sum is less than some value (which you decide, say T), then the output is zero.
- Otherwise, the output is a one.

M-P neuron learning model



- The variables w1, w2 and w3 indicate which input is excitatory, and which one is inhibitory.
- These are called "weights".
- In this model, if a weight is 1, it is an excitatory input.
- If it is -1, it is an inhibitory input.
- x1, x2, and x3 represent the inputs.
- There could be more (or less) inputs if required.

- Accordingly, there would be more 'w's to indicate if that particular input is excitatory or inhibitory.
- You can calculate the sum using the 'x's and 'w's... something like this:
- sum = $x_1w_1 + x_2w_2 + x_3w_3 + ...$
- This is what is called a 'weighted sum'.
- the sum has been calculated, we check if sum < T or not. If it is, then the output is made zero. Otherwise, it is made a one.

M-P neuron learning model

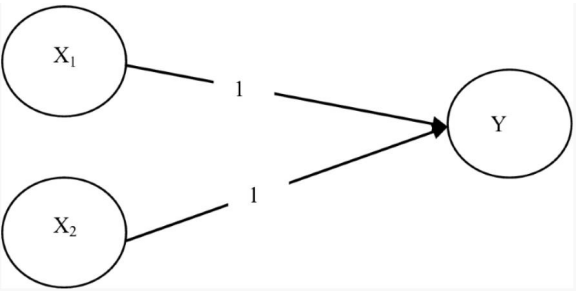
$$loss = \sum_i (y_i - \hat{y}_i)^2$$

- y_i: the actual output in the training data
- y_i: the output of the M-P neuron

- Inhibitory inputs are those that have maximum effect on the decision making irrespective of other inputs.
- if at least one inhibitory input is 1 then the output will always be 0.
- Learning in MP Neuron consists of finding the threshold value with lowest error for prediction.
- This is done with brute force for a single parameter.
- The squared loss function is applied.
- It finds the difference between the predicted value and the actual prediction as a square.

M-P neuron learning model example

X1	X2	Y
1	1	1
1	0	0
0	1	0
0	0	0



Algorithm: M-P neuron learning algorithm

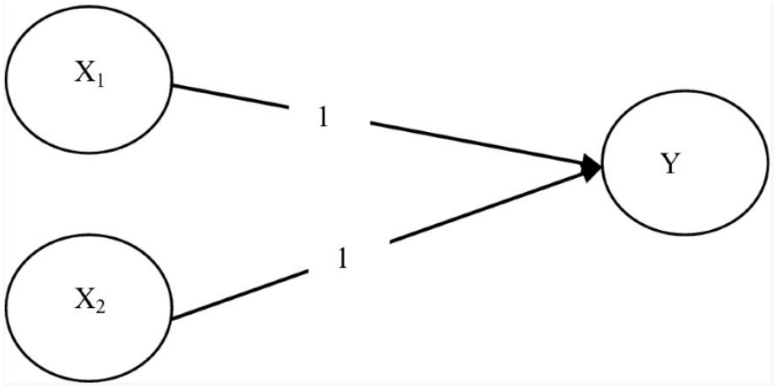
```
X1:=[1100]
X2:=[1010]
Y:=[1000]
total_loss_function_value:=0
 for i=1:4
     weighted_sum:= X1(i)*w1+X2(i)*w2
      if (weighted_sum>=T)
           Y'(i)=1
      else
           Y'(i)=0
      end
      loss_function_value:=(Y(i)-Y'(i))*(Y(i)-Y'(i))
      total_loss_function_value:=total_loss_function_value+loss_function_value
end
```

- Generate the Output of Logic **AND** Function by McCulloch–Pitts Neuron Model

- The algorithm is run for different values of T to find the lowest total_loss_function_value and T which proivides the lowest total_loss_function_value will be threshold value for trained neuron.
- The Threshold will be equal to 2.

M-P neuron learning model example

X1	X2	Y
1	1	1
1	0	1
0	1	1
0	0	0

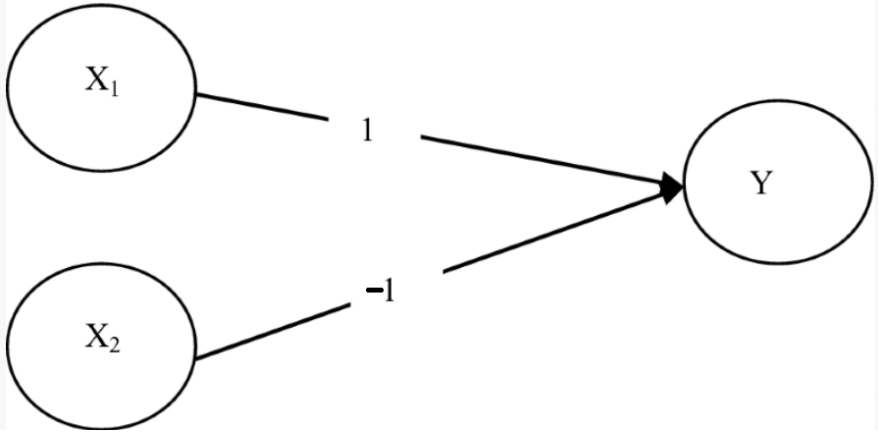


- Threshold value will be 1.

- Generate the Output of Logic **OR** Function by McCulloch–Pitts Neuron Model

M-P neuron learning model example

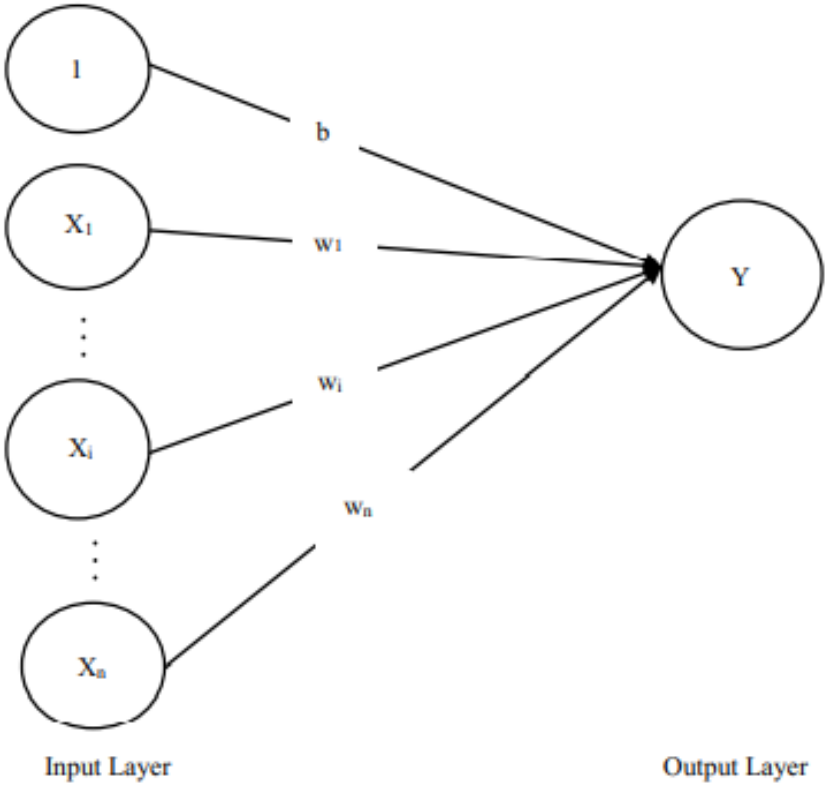
X1	X2	Y
1	1	0
1	0	1
0	1	0
0	0	0



- Threshold value will be 1.

- Generate the Output of Logic **AND NOT** Function by McCulloch–Pitts Neuron Model

Hebbian learning network



- Architecture of a Hebbian network

- Hebbian learning rule is one of the earliest and the simplest learning rules for the neural networks (1994).
- It was proposed by Donald Hebb.
- Hebbian network is a single layer neural network which consists of one input layer with many input units and one output layer with one output unit.
- This architecture is usually used for pattern classification

Hebbian learning network

The step-by-step algorithm of Hebbian learning rule is as follows (Sivanandam et al. 2006):

Step 1: Initialize all weights and bias to zero, i.e., $w_i = 0$ for $i = 1$ to n, $b = 0$. Here, n is the number of input neurons.
Step 2: For each input training vector and target output pair, **S** : t, do steps 2–5.
Step 3: Set activation for input units: $x_i = \mathbf{S}_i$, $i = 1, \ldots, n$.
Step 4: Set activation for output unit: $y = t$.
Step 5: Adjust the weights and bias:

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \text{ for } i = 1, \ldots, n,$$
$$b(\text{new}) = b(\text{old}) + y.$$

If the bias is considered to be an input signal that is always 1, the weight change can be written as

$$w(\text{new}) = w(\text{old}) + \Delta w, \text{ where } \Delta w = xy.$$

Write a Matlab Program to Train XOR Function Using Hebbian Learning with Bipolar Input and Targets

- **Program**:
- % Hebbian Algorithm
- % XOR Gate
- clear all
- clc
- x1=[1 1 -1 -1];
- x2=[1 -1 1 -1];
- b=[1 1 1 1];
- y=[-1 1 1 -1];
- w1=0;
- w2=0;
- h=0;
- for i=1:4
- d1(i)=x1(i)*y(i);
- d2(i)=x2(i)*y(i);
- db(i)=b(i)*y(i);
- w1_n(i)=w1+d1(i);
- w2_n(i)=w2+d2(i);
- b_n(i)=h+db(i);
- w1=w1_n(i);
- w2=w2_n(i);
- h=b_n(i);
- end
- disp(' Input Target Weight Change Weights');
- disp(' x1 x2 b Y dw1 dw2 db w1 w2 B');
- H=[x1' x2' b' y' d1' d2' db' w1_n' w2_n' b_n'];
- disp(H)

Output:

Inputs			Target	Weight Change			Weights		
x1	x2	b	Y	dw1	dw2	db	w1	w2	B
1	1	1	-1	-1	-1	-1	-1	-1	-1
1	-1	1	1	1	-1	1	0	-2	0
-1	1	1	1	-1	1	1	-1	-1	1
-1	-1	1	-1	1	1	-1	0	0	0

Machine Learning Algorithms

- There is a wide variety of machine learning algorithms that can be grouped in three main categories:
- **Supervised learning algorithms** model the relationship between features (independent variables) and a label (target) given a set of observations.
- Then the model is used to predict the label of new observations using the features.
- Depending on the characteristics of target variable, it can be a classification (discrete target variable) or a regression (continuous target variable) task.
- Examples of Supervised Learning: Regression, Decision Tree, Random Forest, KNN, Logistic Regression etc.

Machine learning algorithms

- **Unsupervised learning algorithms** try to find the structure in unlabeled data.
- In this algorithm, we do not have any target or outcome variable to predict / estimate. It is used for clustering population in different groups, which is widely used for segmenting customers in different groups for specific intervention.
- Examples of Unsupervised Learning: Apriori algorithm, K-means.

Machine learning algorithms

- **Reinforcement learning** works based on an action-reward principle.
- An agent learns to reach a goal by iteratively calculating the reward of its actions.
- This agent learns from past experience and tries to capture the best possible knowledge to make accurate business decisions. Example of Reinforcement Learning: Markov Decision Process

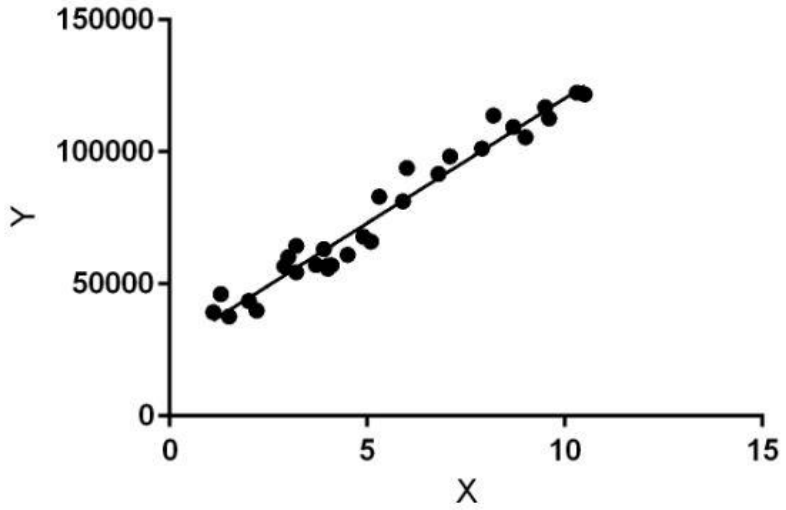
Linear Regression

- It is used to estimate real values (cost of houses, number of calls, total sales etc.) based on continuous variable(s). Here, we establish relationship between independent and dependent variables by fitting a best line.
- This best fit line is known as regression line and represented by a linear equation
- Y= a *X + c.
- We have seen equation like below in maths classes. y is the output we want. x is the input variable. c = constant and a is the slope of the line.
- y = c + ax
- c = constant
- a = slope

Linear Regression

- Regression models a target prediction value based on independent variables.
- It is mostly used for finding out the relationship between variables and forecasting.
- Different regression models differ based on – the kind of relationship between dependent and independent variables, they are considering and the number of independent variables being used.
- Linear regression performs the task to predict a dependent variable value (y) based on a given independent variable (x). So, this regression technique finds out a linear relationship between x (input) and y(output).
- Hence, the name is Linear Regression.

Linear Regression



- In the figure above, X (input) is the work experience and Y is output.
- The regression line is the best fit line for our model.

Hypothesis for Linear Regression

- Y=aX+c
- While training the model we are given :
- x: input training data (univariate – one input variable(parameter))
- y: labels to data (supervised learning)

- When training the model – it fits the best line to predict the value of y for a given value of x.
- The model gets the best regression fit line by finding the best c and a values.
- c: intercept
- a: coefficient of x

Hypothesis for Linear Regression

- Once we find the best a and c values, we get the best fit line. So when we are finally using our model for prediction, it will predict the value of y for the input value of x.

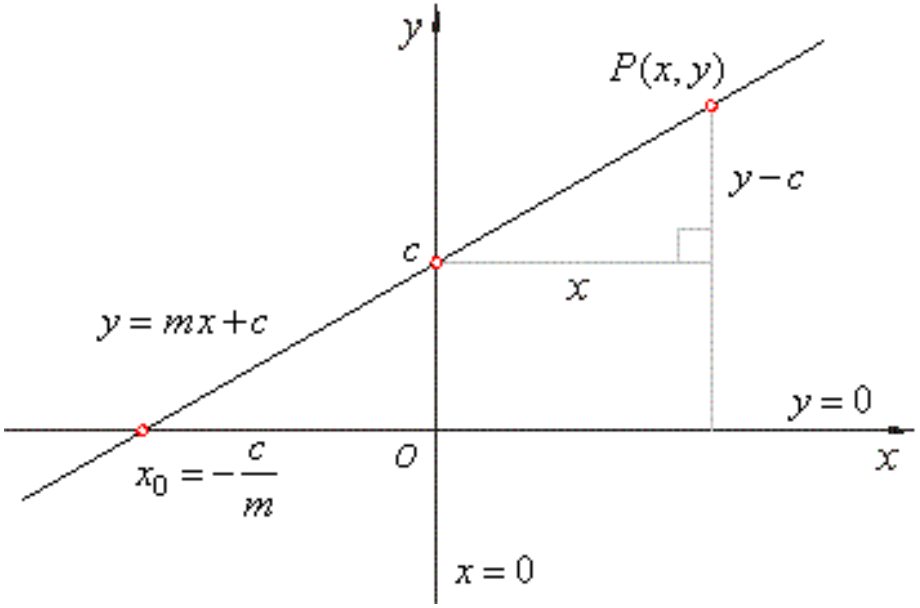
- **How to update a and c values to get the best fit line ?**
- **Cost Function (J):**
- By achieving the best-fit regression line, the model aims to predict y value such that the error difference between predicted value and true value is minimum.
- So, it is very important to update the a and c values, to reach the best value that minimize the error between predicted y value (pred) and true y value (y).

Hypothesis for Linear Regression

$$minimize \frac{1}{n}\sum_{i=1}^{n}(pred_i - y_i)^2 \qquad J = \frac{1}{n}\sum_{i=1}^{n}(pred_i - y_i)^2$$

- Cost function(J) of Linear Regression is the **Mean Squared Error (MSE)** between predicted y value (pred) and true y value (y).
- **Gradient Descent:** To update a and c values in order to reduce Cost function (minimizing **MSE** value) and achieving the best fit line the model uses **Gradient Descent.**
- The idea is to start with random a and c values and then iteratively updating the values, reaching minimum cost.
- A learning rate is used as a scale factor and the coefficients are updated in the direction towards minimizing the error.
- The process is repeated until a minimum sum squared error is achieved or no further improvement is possible.

Linear Regression Implementation



- Let **X** be the independent variable and **Y** be the dependent variable.
- We will define a linear relationship between these two variables as follows:

$$Y = mX + c$$

- m is the slope of the line and c is the y intercept.
- We will use this equation to train our model with a given dataset and predict the value of Y for any given value of X.
- Our challenge is to determine the value of m and c, such that the line corresponding to those values is the best fitting line or gives the minimum error.

Linear Regression Implementation

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)^2$$

Mean Squared Error Equation

- The loss is the error in our predicted value of m and c. Our goal is to minimize this error to obtain the most accurate value of m and c.
- We will use the Mean Squared Error function to calculate the loss.

There are three steps in this function:

- Find the difference between the actual y and predicted y value(y = mx + c), for a given x.
- Square this difference.
- Find the mean of the squares for every value in X.

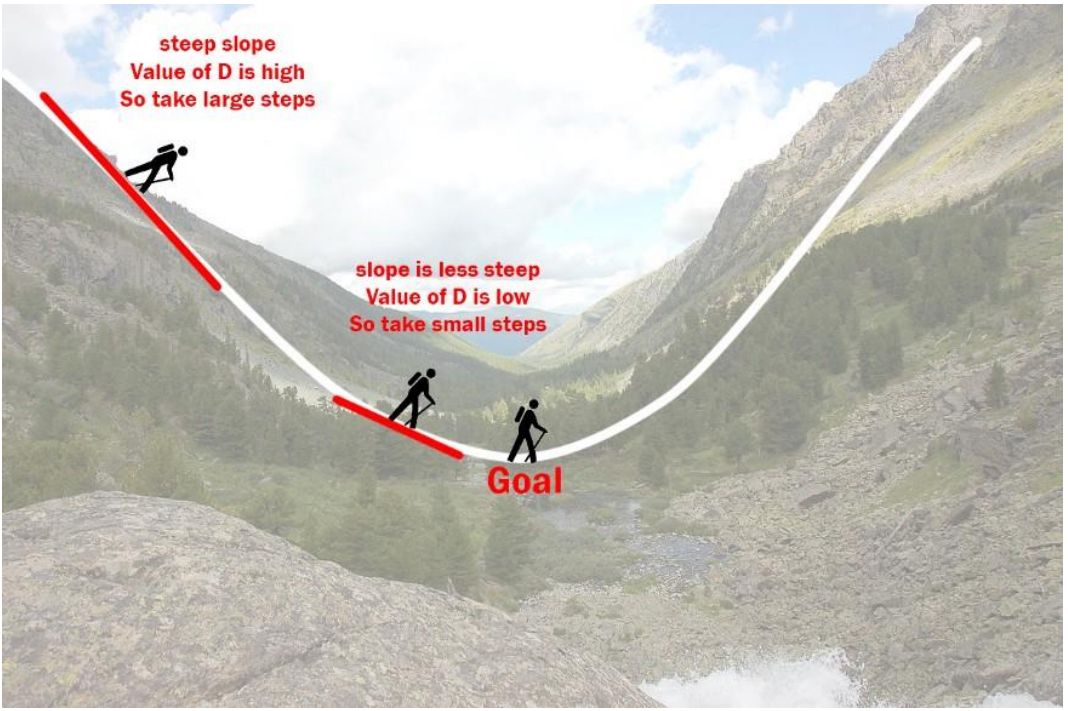
Linear Regression Implementation

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

Substituting the value of $\bar{y}_i$

- Lets substitute the value of $\bar{y}_i$.
- Here y_i is the actual value and $\bar{y}_i$ (i.e., mxi+c) is the predicted value.
- So we square the error and find the mean, hence the name Mean Squared Error.
- Now that we have defined the loss function, lets get into the interesting part — minimizing it and finding m and c.

Linear Regression Implementation



- Illustration of how the gradient descent algorithm works

- Imagine a valley and a person with no sense of direction who wants to get to the bottom of the valley.
- He goes down the slope and takes large steps when the slope is steep and small steps when the slope is less steep.
- He decides his next position based on his current position and stops when he gets to the bottom of the valley which was his goal.

1. Initially let m = 0 and c = 0. Let L be our learning rate. This controls how much the value of **m** changes with each step. L could be a small value like 0.0001 for good accuracy.

2. Calculate the partial derivative of the loss function with respect to m, and plug in the current values of x, y, m and c in it to obtain the derivative value **D**.

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$

Derivative with respect to m

D_m is the value of the partial derivative with respect to **m**. Similarly lets find the partial derivative with respect to **c**, Dc :

$$D_c = \frac{-2}{n}\sum_{i=0}^{n} (y_i - \bar{y}_i)$$

Derivative with respect to c

3. Now we update the current value of **m** and **c** using the following equation:

$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

4. We repeat this process until our loss function is a very small value or ideally 0 (which means 0 error or 100% accuracy). The value of **m** and **c** that we are left with now will be the optimum values.

Linear Regression Implementation

- Now going back to our analogy, m can be considered the current position of the person.
- D is equivalent to the steepness of the slope and L can be the speed with which he moves.
- Now the new value of m that we calculate using the above equation will be his next position, and L×D will be the size of the steps he will take.
- When the slope is more steep (D is more) he takes longer steps and when it is less steep (D is less), he takes smaller steps.
- Finally he arrives at the bottom of the valley which corresponds to our loss = 0.
- Now with the optimum value of m and c our model is ready to make predictions !

Implementing Linear Regression

```
# Making the imports

import numpy as np

import pandas as pd


# Preprocessing Input data

data = pd.read_csv('data.csv')
```

Implementing Linear Regression

```
# Building the model
m = 0
c = 0

L = 0.0001  # The learning Rate
epochs = 1000  # The number of iterations to perform gradient descent

n = float(len(X)) # Number of elements in X

# Performing Gradient Descent
for i in range(epochs):
    Y_pred = m*X + c  # The current predicted value of Y
    D_m = (-2/n) * sum(X * (Y - Y_pred))  # Derivative wrt m
    D_c = (-2/n) * sum(Y - Y_pred)  # Derivative wrt c
    m = m - L * D_m  # Update m
    c = c - L * D_c  # Update c

print (m, c)
```

Implementing Linear Regression

```
# Making predictions

Y_pred = m*X + c
```

Gradient descent is one of the simplest and widely used algorithms in machine learning, mainly because it can be applied to any function to optimize it